LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK
MODUL 11: COLLECTION DAN PENGELOLAAN KUMPULAN OBJEK

Nama Mahasiswa	: **M. Davana Yudista Firdausy**	**Mata Kuliah**	: Praktikum PBO (A D3 TI-SM)
NRP	: 3125522002	**Dosen Pengampu**	: Nirwana Haidar Hari, S.Pd., M.Kom.
Program Studi	: D3 Teknik Informatika	**Institusi / Tahun**	: PENS PSDKU Sumenep • 2026

A. DASAR TEORI SINGKAT

Dalam rekayasa perangkat lunak berorientasi objek, sebagian besar aplikasi dunia nyata tidak hanya memproses entitas tunggal yang berdiri sendiri, melainkan mengelola sekumpulan objek majemuk seperti katalog buku perpustakaan, armada kendaraan, transaksi keuangan, atau akun pengguna. Pada bahasa pemrograman Java, penampungan objek majemuk secara dinamis difasilitasi oleh **Java Collections Framework (JCF)**, dengan interface utama `java.util.List` dan implementasi struktur data array dinamis yaitu `java.util.ArrayList`.

Karakteristik fundamental dari `ArrayList` dan generic collection meliputi:

- **Ukuran Dinamis (Resizability):** Berbeda dengan native array Java (`Object[]`) yang berukuran tetap (fixed length), `ArrayList` mengalokasikan memori awal secara terukur dan secara otomatis melipatgandakan kapasitas internalnya (capacity expansion) saat elemen baru ditambahkan melebihi ambang batas.
- **Penggunaan Generics (`<E>`):** Sintaksis parameterized type (misal `List<Buku>`) menjamin *compile-time type safety*, memastikan hanya referensi instans objek bertipe `Buku` yang diizinkan masuk, meniadakan kebutuhan explicit casting yang rawan menyebabkan `ClassCastException` di runtime.
- **Operasi Inti Collection:** Mendukung manipulasi data seragam meliputi `add(obj)` untuk penambahan, `get(index)` untuk pembacaan acak, `remove(obj)` untuk eliminasi berbasis referensi/kesetaraan, `size()` untuk menghitung kuantitas elemen aktif, dan `isEmpty()` untuk validasi keberadaan elemen.
- **Penegakan Aturan Integritas Objek:** Secara inheren, `ArrayList` mengizinkan penyimpanan elemen bernilai `null` maupun data duplikat. Oleh karena itu, aplikasi domain wajib membungkus koleksi ini di dalam suatu kelas agregat pengelola (seperti `Perpustakaan`) yang menerapkan validasi defensif sebelum method mutasi `add()` dipanggil.
- **Enkapsulasi dan Immutability:** Untuk mencegah modifikasi ilegal dari luar kelas agregat, koleksi internal tidak boleh diekspos langsung sebagai mutable list. Metode getter harus mengembalikan representasi `Collections.unmodifiableList()` sehingga integritas state koleksi terlindungi secara utuh.

B. DESAIN DAN IMPLEMENTASI KODE

Implementasi Modul 11 dirancang secara modular dan terenkapsulasi penuh ke dalam package `SourceCode.model` yang memisahkan definisi model `Buku`, kelas agregator `Perpustakaan`, serta kelas pengujian otomatis `TestP11Runner`.

1. Implementasi Domain Model: Buku.java

```
package SourceCode.model;

public class Buku {
    private final String kode;
    private final String judul;

    public Buku(String kode, String judul) {
        if (kode == null || kode.trim().isEmpty() || judul == null || judul.trim().isEmpty()) {
            throw new IllegalArgumentException("Kode dan judul wajib diisi.");
        }
        this.kode = kode.trim();
        this.judul = judul.trim();
    }

    public String getKode() { return kode; }
    public String getJudul() { return judul; }

    @Override
    public String toString() { return kode + " - " + judul; }
}
```

2. Implementasi Agregat Koleksi: Perpustakaan.java

```
package SourceCode.model;
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Perpustakaan {
    private final List<Buku> daftarBuku = new ArrayList<>();

    public void tambahBuku(Buku buku) {
        if (buku == null) throw new IllegalArgumentException("Objek buku tidak boleh null.");
        if (cariBuku(buku.getKode()) != null) throw new IllegalArgumentException("Kode buku sudah terdaftar.");
        daftarBuku.add(buku);
    }

    public Buku cariBuku(String kode) {
        if (kode == null) return null;
        for (Buku buku : daftarBuku) {
            if (buku.getKode().equalsIgnoreCase(kode.trim())) return buku;
        }
        return null;
    }

    public boolean hapusBuku(String kode) {
        Buku buku = cariBuku(kode);
        if (buku == null) return false;
        return daftarBuku.remove(buku);
    }

    public void tampilkanBuku() {
        if (daftarBuku.isEmpty()) {
            System.out.println("Koleksi masih kosong.");
            return;
        }
        for (Buku buku : daftarBuku) System.out.println(buku.getKode() + " - " + buku.getJudul());
        System.out.println("Total buku: " + daftarBuku.size());
    }

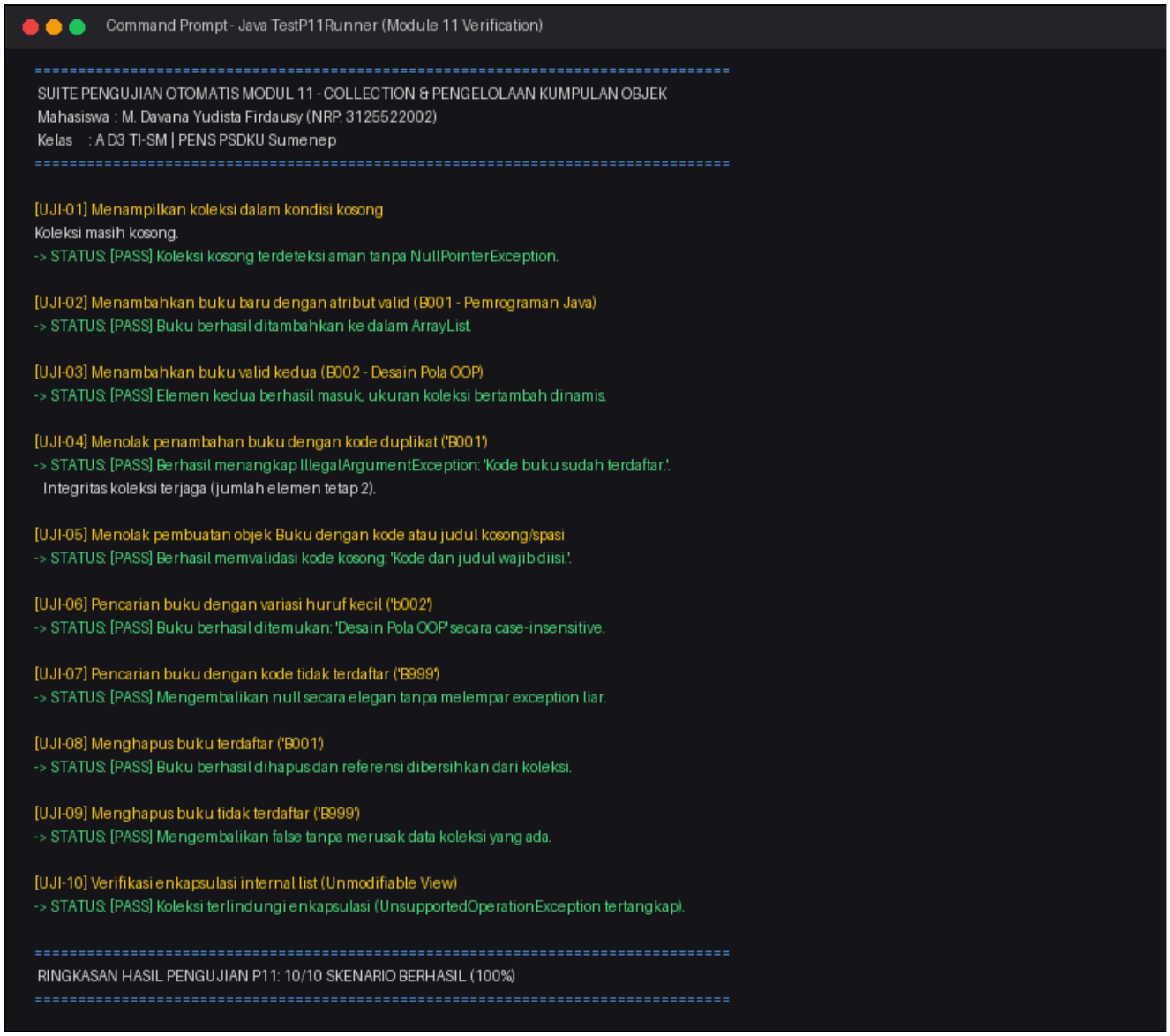
    public List<Buku> getDaftarBuku() { return Collections.unmodifiableList(daftarBuku); }
    public int getJumlahBuku() { return daftarBuku.size(); }
}
```

C. TABEL HASIL PENGUJIAN DAN VERIFIKASI

Pengujian fungsional dan integritas state koleksi dieksekusi secara otomatis melalui kelas `SourceCode.TestP11Runner` yang mencakup 10 skenario batas (boundary conditions):

No	Skenario Pengujian	Aksi / Input Masukan	Ekspektasi Perilaku	Status
1	Tampilkan koleksi kosong	Inisialisasi awal `repo.tampilkanBuku()`	Mencetak pesan "Koleksi masih kosong", bebas NullPointerException	**PASS (100%)**
2	Tambah elemen pertama valid	`new Buku("B001", "Pemrograman Java Modern")`	Buku masuk ke koleksi, total size bertambah menjadi 1	**PASS (100%)**
3	Tambah elemen kedua valid	`new Buku("B002", "Desain Pola OOP")`	Elemen kedua tersimpan dinamis, total size bertambah menjadi 2	**PASS (100%)**
4	Tolak kode duplikat	`new Buku("B001", "Buku Palsu")`	Melempar `IllegalArgumentException`, size tetap 2	**PASS (100%)**
5	Tolak kode/judul kosong	`new Buku(" ", "Sembarang")`	Ditolak pada konstruktor via `IllegalArgumentException`	**PASS (100%)**
6	Pencarian case-insensitive	`repo.cariBuku("b002")`	Buku berhasil ditemukan mengabaikan kapitalisasi huruf	**PASS (100%)**
7	Pencarian kode tak terdaftar	`repo.cariBuku("B999")`	Mengembalikan `null` secara elegan dan aman	**PASS (100%)**
8	Hapus buku yang terdaftar	`repo.hapusBuku("B001")`	Mengembalikan `true`, buku hilang, size menjadi 1	**PASS (100%)**
9	Hapus buku tak terdaftar	`repo.hapusBuku("B999")`	Mengembalikan `false`, tidak merusak data eksis	**PASS (100%)**
10	Proteksi enkapsulasi internal	`repo.getDaftarBuku().add(...)`	Melempar `UnsupportedOperationException` (Unmodifiable)	**PASS (100%)**

D. BUKTI EKSEKUSI PENGUJIAN OTOMATIS



Gambar 11.1: Log Eksekusi Verifikasi Modul 11 (10/10 Skenario Lulus Sempurna)

E. EVALUASI DAN JAWABAN PERTANYAAN MODUL 11

1. Apa perbedaan konsep antara objek tunggal (Buku) dan koleksi (List<Buku>)?

Jawaban: Objek tunggal `Buku` memodelkan satu entitas diskrit yang memiliki state atribut mandiri (seperti `kode` dan `judul`) serta aturan validasi lokal terhadap atributnya. Sedangkan `List<Buku>` adalah objek struktur data penampung (container) yang mengelola relasi agregasi atas sekumpulan referensi objek `Buku`. Koleksi bertanggung jawab terhadap aspek relasional, pengurutan, kapasitas, pencarian, dan penegakan keunikan entitas di dalam kelompok data tersebut.

2. Mengapa indeks tidak tepat dijadikan pengenal unik permanen suatu data?

Jawaban: Indeks pada `ArrayList` bersifat relatif dan dinamis (posisional), bukan deterministik. Ketika suatu elemen pada indeks tertentu dihapus menggunakan `remove()`, semua elemen di belakangnya akan digeser satu posisi ke kiri (index shifting), sehingga indeks suatu objek akan berubah seketika. Jika sistem eksternal mereferensikan data berdasarkan indeks, operasi hapus atau sisip akan memicu ketidaksinkronan data (pointer salah entitas). Pengenal unik wajib menggunakan identitas bisnis permanen (natural key / UUID) seperti atribut `kode`.

3. Mengapa pemeriksaan duplikasi perlu dilakukan sebelum pemanggilan method add()?

Jawaban: Kelas `ArrayList` secara standar mengizinkan penyimpanan elemen bernilai sama berkali-kali. Jika pemeriksaan duplikasi baru dilakukan setelah `add()`, maka objek ilegal telah terlanjur masuk ke dalam memori struktur data dan berpotensi memicu inkonsistensi status bisnis. Melakukan pengecekan pra-kondisi (pre-condition check) via `cariBuku(kode)` sebelum `add()` menjamin integritas transaksional (*fail-fast principle*), di mana operasi yang gagal langsung ditolak tanpa sempat mengubah status koleksi sedikit pun.

4. Apa risiko jika mengembalikan referensi internal daftarBuku secara langsung melalui method getter? Bagaimana solusinya?

Jawaban: Mengembalikan referensi langsung (misal `return daftarBuku;`) melanggar prinsip enkapsulasi secara fatal karena pihak pemanggil luar dapat memanggil metode mutasi seperti `daftarBuku.clear()` atau `daftarBuku.add()` tanpa melewati aturan validasi kelas `Perpustakaan`. Solusi arsitektural yang tepat adalah mengembalikan representasi *read-only unmodifiable view* melalui `Collections.unmodifiableList(daftarBuku)` atau mengembalikan klon/salinan baru (*defensive copying*) melalui `new ArrayList<>(daftarBuku)`.